

Performance Optimization

Make Your App Fast, Efficient and Scalable

■ LEARNING OUTCOME

By the end of this module you will measure and optimise frontend and backend performance — achieving Lighthouse scores above 90, reducing API response times, optimising database queries, and implementing caching strategies.

■ Skills You Will Gain

- Measure performance with Lighthouse, WebPageTest, and React Profiler
- Optimise images with modern formats and lazy loading
- Implement code splitting and lazy loading for React
- Add server-side caching with Redis or in-memory cache
- Optimise database queries with EXPLAIN and indexes
- Implement CDN and HTTP caching headers

■ Performance IS user experience. A 1-second delay reduces conversions by 7%. Google uses Core Web Vitals as a ranking factor. Fast apps keep users; slow apps lose them. Performance is not optional -- it is a feature.

SECTION 1

Measuring Performance

Tool	What It Measures	How to Access
Lighthouse	Performance, Accessibility, SEO, Best Practices scores	Chrome DevTools F12 > Lighthouse tab
Core Web Vitals	LCP (loading), FID/INP (interactivity), CLS (visual stability)	PageSpeed Insights: pagespeed.web.dev
WebPageTest	Waterfall chart, time to first byte, render timeline	webpagetest.org -- use real devices
Network tab	Request sizes, waterfall, blocking resources	Chrome DevTools F12 > Network tab
React Profiler	Component render times, what caused re-renders	React DevTools > Profiler tab
Bundle Analyser	What's in your JS bundle and what's bloating it	<code>npx vite-bundle-visualizer</code>
EXPLAIN ANALYZE	PostgreSQL query execution plan and actual timing	Run in Prisma Studio or <code>psql</code>

SECTION 2

Frontend Performance

```
# IMAGE OPTIMISATION -- biggest performance win # Convert to WebP: npx sharp-cli input.jpg -o
output.webp # Compress: use squoosh.app, TinyPNG, or imagemin <!-- Responsive images with srcset
--> <img src='hero-800.webp' srcset='hero-400.webp 400w, hero-800.webp 800w, hero-1200.webp 1200w'
sizes='(max-width: 600px) 100vw, (max-width: 1024px) 80vw, 1200px' alt='Hero image' width='1200'
height='600' loading='lazy' <!-- lazy load below-fold images --> decoding='async' <!-- don't block
rendering while decoding --> > // REACT CODE SPLITTING -- split bundle by route import { lazy,
Suspense } from 'react'; // Each lazy import becomes a separate chunk // Only downloaded when the
user navigates to that route const Dashboard = lazy(() => import('./pages/Dashboard')); const
Settings = lazy(() => import('./pages/Settings')); const Admin = lazy(() =>
import('./pages/Admin')); function App() { return ( <Suspense fallback={<PageSkeleton />} > <Routes>
<Route path='/dashboard' element={<Dashboard />} /> <Route path='/settings' element={<Settings />}
/> <Route path='/admin' element={<Admin />} /> </Routes> </Suspense> ); } // VIRTUALISATION --
render only visible items in long lists // npm install @tanstack/react-virtual import {
useVirtualizer } from '@tanstack/react-virtual'; function VirtualList({ items }) { const parentRef
= useRef(null); const virt = useVirtualizer({ count: items.length, getScrollElement: () =>
parentRef.current, estimateSize: () => 64, // estimated row height in px }); return ( <div
ref={parentRef} style={{ height: '600px', overflow: 'auto' }}> <div style={{ height:
virt.getTotalSize() }}> {virt.getVirtualItems().map(row => ( <div key={row.index} style={{
position: 'absolute', top: row.start, height: row.size, width: '100%' }}> <TaskRow
task={items[row.index]} /> </div> ))} </div> </div> ); } // FONT OPTIMISATION /* In HTML head --
preconnect reduces DNS + TCP time */ <link rel='preconnect' href='https://fonts.googleapis.com'>
<link rel='preconnect' href='https://fonts.gstatic.com' crossorigin> /* font-display: swap -- shows
fallback font immediately */ @font-face { font-display: swap; }
```

SECTION 3

Backend Performance and Caching

```
// REDIS CACHING -- store frequently accessed data in memory npm install redis // config/redis.js
const { createClient } = require('redis'); const redis = createClient({ url: process.env.REDIS_URL
}); await redis.connect(); module.exports = redis; // middleware/cache.js const redis =
require('../config/redis'); const cache = (ttlSeconds = 60) => async (req, res, next) => { const
key = `cache:${req.originalUrl}`; const cached = await redis.get(key); if (cached) {
res.setHeader('X-Cache', 'HIT'); return res.json(JSON.parse(cached)); } // Override res.json to
cache the response const originalJson = res.json.bind(res); res.json = (data) => { redis.setEx(key,
ttlSeconds, JSON.stringify(data)); res.setHeader('X-Cache', 'MISS'); return originalJson(data); };
next(); }; // Apply caching to GET routes (not mutations!) router.get('/', cache(300),
tasksCtrl.getAll); // cache 5 minutes router.get('/:id', cache(60), tasksCtrl.getOne); // cache 1
minute // Invalidate cache after mutations exports.update = async (req, res) => { const task =
await taskService.update(req.params.id, req.body); await redis.del(`cache:/api/tasks`); //
invalidate list cache await redis.del(`cache:/api/tasks/${task.id}`); // invalidate item cache
res.json(task); }; // HTTP CACHING HEADERS -- browser caches static assets // In Express: serve
static files with cache headers app.use('/static', express.static('public', { maxAge: '1y', //
cache static assets for 1 year (they have hashed names) etag: true, lastModified: true, })); // API
responses: no-cache for dynamic data app.use('/api', (req, res, next) => {
res.setHeader('Cache-Control', 'no-cache, no-store, must-revalidate'); next(); }); // DATABASE
QUERY OPTIMISATION // Use EXPLAIN ANALYZE to see query plan -- Run in PostgreSQL to see slow
queries: EXPLAIN ANALYZE SELECT * FROM tasks WHERE user_id = 'abc' AND status = 'todo' ORDER BY
created_at DESC LIMIT 20; -- Look for: Seq Scan (bad on large tables) -> needs index -- Add index:
CREATE INDEX CONCURRENTLY idx_tasks_user_status ON tasks(user_id, status); // Prisma N+1 problem
and fix // BAD: N+1 queries (1 for list + N for each user) const tasks = await
prisma.task.findMany(); for (const task of tasks) { const user = await prisma.user.findUnique({
where:{id:task.userId} }); // N queries! } // GOOD: use include for JOIN in one query const tasks =
await prisma.task.findMany({ include: { user: { select: { name:true, email:true } } }, // one query
});
```

■ PRACTICE Q & A

Q1. What are Core Web Vitals and why do they matter?

A: *LCP (Largest Contentful Paint): how long until main content loads (<2.5s good). INP (Interaction to Next Paint): response time to user input (<200ms good). CLS (Cumulative Layout Shift): unexpected layout shifts (<0.1 good). Google uses these as ranking signals.*

Q2. What is the N+1 query problem?

A: *Fetching a list of N items, then making N separate database queries for related data. Example: fetch 20 tasks, then 20 separate queries for each task's user. Fix: use JOIN (include in Prisma) to fetch all data in one query.*

Q3. What is the difference between server caching and HTTP caching?

A: *Server cache (Redis): server stores response data to avoid database queries -- reduces backend processing time. HTTP cache: browser or CDN stores the response to avoid HTTP requests entirely -- reduces network traffic. Use both for maximum performance.*

Q4. What is code splitting and how does it improve performance?

A: *Splitting the JS bundle into smaller chunks loaded on demand. With lazy routes, the Admin page code only downloads when an admin navigates there. Reduces initial bundle size -- users only download code for pages they actually visit.*

Q5. What is virtualisation/windowing for long lists?

A: *Instead of rendering 10,000 DOM elements for a long list, render only the ~20 visible items and swap elements as the user scrolls. @tanstack/react-virtual handles this. Rendering 10,000 DOM nodes causes layout slowness; rendering 20 is instant.*

■ Performance Cheat Sheet	
Lighthouse in Chrome DevTools	Run performance audit
npmx vite-bundle-visualizer	Visualise bundle contents
loading='lazy'	Lazy-load below-fold images

WebP format	30-50% smaller than JPEG
lazy(()=>import('./Page'))	Code-split React routes
useVirtualizer	Virtualise long lists
Redis cache(300)	Cache API response 5 minutes
redis.del('cache:/api/x')	Invalidate cache after mutation
EXPLAIN ANALYZE	Diagnose slow SQL queries
include: {user:true}	Fix N+1 with Prisma JOIN
Cache-Control: max-age=31536000	Cache static assets 1 year
font-display: swap	Show fallback font immediately